testing_doc.pdf

Testing documentation for Homework Assignment 1

1. How the input files were created

The submitted zip should not contain data files, so the automatic tester was designed to create temporary input files at runtime. This still satisfies the requirement that the tester load input files, because it writes a movies file and a ratings file to a temporary directory, then loads them through the same parsing functions used by the main program.

The test data in this draft was generated by ChatGPT and embedded directly inside test_movie_recommender.py as string literals. No permanent external data files are required for the tester to run.

2. Size of each generated test file

Test file	Total rows	Valid rows	Invalid rows
movies.txt	10	7	3
ratings.txt	17	12	5

3. How possible input problems were incorporated

- The movies file includes one row with a missing movie ID, one malformed row with too few fields, and one duplicate movie title.
- The ratings file includes one rating for an unknown movie, one non-numeric rating, one out-of-range rating, one duplicate rating by the same user for the same movie, and one malformed line.
- These bad rows verify that the code does not crash on imperfect input and instead skips invalid rows while recording warnings.

4. How the data distribution exercises multiple code paths

- Three genres are present: Action, Comedy, and Drama.
- Action contains multiple movies that tie at the same average rating, which verifies tie-breaking behavior.
- The overall movie ranking also contains ties, which exercises alphabetical secondary sorting.
- User u1 rates both Action and Comedy, allowing the tester to verify the user-preferred-genre calculation.
- User u1 has already rated the top two Action movies, leaving only one eligible recommendation; this checks the filter that removes already-rated movies from recommendations.
- An unknown user is also tested to confirm that the recommendation function returns an empty list when the user has no prior ratings.

5. Expected results covered by the tester

The tester checks all major requirements: file loading, invalid-row handling, top n movies, top n movies by genre, top n genres, user preferred genre, recommendations for a valid user, and recommendations for a user with no ratings. Each check prints the expected value, the actual value, and a PASS/FAIL result so correctness is visible without manually navigating the CLI.

6. Final testing summary

This approach provides broad coverage while keeping the submission clean. Because the test data is generated inside the tester itself, the final hw1.zip can include only the four required files and still allow the grader or student to run the tests immediately.